

LABORATORY

LAB 18

CAN Loopback Communication

Internal CAN Transmission and Reception with EduFramework

S32K144 · MaaZEDU

Contents

1	Introduction	2
1.1	Overview	2
1.2	Objectives	2
2	Background	2
2.1	CAN in Automotive Systems	2
2.2	Classical CAN and Data Frame Structure	2
2.3	CAN IDs, Bus Arbitration, and Error Detection	3
2.4	Internal Loopback	3
3	Hardware Setup	4
3.1	Required Hardware	4
3.2	Interfaces Used	4
3.3	Power Configuration and Connections	4
4	EduFramework API	5
4.1	<code>CAN_Frame_t</code>	5
4.2	<code>CAN_begin()</code>	5
4.3	<code>CAN_setMode()</code>	5
4.4	<code>CAN_send()</code>	6
4.5	<code>CAN_available()</code> and <code>CAN_read()</code>	6
5	Lab Exercise	7
5.1	Requirements	7
5.2	Program	7
5.3	Verification	9
6	Extension	9
6.1	Extension Exercise	10
7	References	10

1 Introduction

1.1 Overview

Controller Area Network (CAN) is a serial communication protocol developed for systems with distributed controllers, particularly in automotive applications. Rather than requiring a separate connection between every pair of Electronic Control Units (ECUs), multiple devices can exchange messages over a shared CAN network [1] [2].

This lab introduces Classical CAN and demonstrates how to construct, transmit, and receive a CAN frame using the Arduino-style CAN API of EduFramework on the S32K144. Internal Loopback mode allows the FlexCAN controller to receive its own transmitted frame, enabling verification on a single board without a second ECU [6]. Transmitted and received values are displayed on the Serial Monitor. The exercise focuses on CAN IDs, frame formats, data length, and payload; physical CAN bus communication between two ECUs will be covered in Lab 19.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the role of CAN in communication networks connecting vehicle ECUs.
2. Distinguish standard 11-bit and extended 29-bit CAN IDs and identify the Classical CAN payload limit.
3. Identify the principal fields of a CAN data frame and explain message priority.
4. Explain Internal Loopback and the scope of verification provided by this mode.
5. Use EduFramework CAN APIs to transmit, receive, and compare data on the Serial Monitor.

2 Background

2.1 CAN in Automotive Systems

A vehicle may contain multiple ECUs responsible for functions such as engine management, powertrain control, braking, and information display. These ECUs must exchange status information and control data without an excessive increase in wiring. CAN provides a shared communication mechanism for nodes on a network [1] [2]. CAN is a message-oriented protocol. Each message carries an identifier (CAN ID) rather than a fixed destination device address. On a conventional CAN bus, nodes can observe transmitted messages; the application or controller determines which messages should be processed [4].

At the physical layer, high-speed CAN uses two differential signal lines, `CAN_H` and `CAN_L`, to exchange data between nodes through CAN transceivers. The CAN controller in the MCU handles the protocol, while the transceiver converts signals between the controller and the physical bus [2].

2.2 Classical CAN and Data Frame Structure

Classical CAN supports two identifier formats: Standard CAN uses an 11-bit CAN ID, whereas Extended CAN uses a 29-bit CAN ID. A Classical CAN data frame contains between 0 and 8 bytes of payload [3].

Format	CAN ID	Characteristics
Standard	11-bit	Identifier range from 0x000 to 0x7FF .
Extended	29-bit	Identifier range from 0x00000000 to 0x1FFFFFFF .

Table 18.2.1. Identifier formats supported by Classical CAN

A CAN data frame contains fields used for synchronization, message identification, data transmission, and error detection. Its main fields are summarized below [3] [4] .

Field	Purpose
SOF	Indicates the beginning of a CAN frame.
Arbitration	Contains the identifier and information used to arbitrate bus access.
Control	Contains control information, including the DLC indicating the data field length in Classical CAN.
Data	Contains a data frame payload of 0 to 8 bytes .
CRC	Enables detection of errors in a transmitted frame.
ACK	Allows a receiving node to acknowledge reception of a valid frame.
EOF	Marks the end of a CAN frame.

Table 18.2.2. Main fields of a Classical CAN data frame

In EduFramework, an application only needs to provide `id` , `format` , `length` , and `data[]` . Protocol fields such as SOF, CRC, and ACK are handled by the CAN controller below the application layer [6] .

2.3 CAN IDs, Bus Arbitration, and Error Detection

When multiple nodes attempt to transmit simultaneously, CAN uses non-destructive arbitration based on message identifiers. For data frames of the same format, the numerically lower identifier has higher priority. A node that loses arbitration stops transmitting and waits to retry when the bus becomes available [3] [4] .

CAN also provides error-detection mechanisms, including CRC checking, bit monitoring, and frame format checking. On a conventional bus, the ACK field informs the transmitter that at least one other node received a valid frame; however, an ACK does not guarantee that the destination ECU's application has processed the data [3] [4] .

Note

Arbitration, CRC, and ACK are introduced to establish an understanding of the CAN protocol. The Loopback exercise does not verify arbitration among multiple ECUs or the operation of the CAN physical layer.

2.4 Internal Loopback

In Normal mode, frames are transmitted through the CAN interface and can be received by other nodes on the bus. Internal Loopback creates an internal reception path within the CAN controller: a transmitted frame is routed back to the controller's receive logic without passing through the external `CAN_H` and `CAN_L` lines [6] . In EduFramework, `CAN_setMode(CAN_MODE_LOOPBACK)` enables this mode. The exercise can therefore verify frame construction, transmission, reception, and receive queue access on a single MCU. A `PASS` result confirms that the compared fields match during the internal test; it does not replace testing on a physical CAN bus.

3 Hardware Setup

3.1 Required Hardware

This lab uses one MaaZEDU Development Board equipped with an S32K144 MCU and FlexCAN0. Because transmission and reception take place in Internal Loopback mode, no second ECU, external CAN transceiver, or CAN_H / CAN_L wiring is required.

Hardware	Description
MaaZEDU Development Board	S32K144 board used to run the CAN Loopback application.
12V DC Adapter	External power supply required for CAN operation on MaaZEDU.
USB Cable	Connection to the computer for programming and Serial Monitor access.

Table 18.3.1. Hardware required for the exercise

3.2 Interfaces Used

EduFramework uses the S32K144 FlexCAN0 peripheral. On MaaZEDU, CAN0_TX is mapped to PTE5 and CAN0_RX to PTE4, as documented for the board [7] [5].

Signal	MCU Pin	Function
CAN0_TX	PTE5	Transmit path from the CAN controller to the transceiver in Normal mode.
CAN0_RX	PTE4	Receive path from the transceiver to the CAN controller in Normal mode.
Serial1	LPUART1	Displays transmitted data, received data, and verification results on the Serial Monitor.

Table 18.3.2. Interfaces relevant to Lab 18

These CAN pins are listed for hardware identification; the Loopback exercise does not require connecting them with wires. Do not short CAN_H to CAN_L, as they are separate lines used for differential CAN communication [2].

3.3 Power Configuration and Connections

According to the MaaZEDU Development Board Guide, the board's CAN function requires an external power source [7]. This lab uses the configuration verified on MaaZEDU: move the power-selection jumper to the external-power position, connect a 12V adapter, and retain the USB connection for programming and Serial Monitor access.

Note

Turn off the power before changing the jumper position. Connect the adapter to the designated board power connector; never apply 12V directly to MCU pins or signal pins. Internal Loopback requires no external CAN wiring. The external-power requirement is specific to the MaaZEDU configuration used in this exercise.

4 EduFramework API

EduFramework provides an Arduino-style CAN API built on the FlexCAN driver. The application does not need to configure Message Buffers, receive interrupts, or the receive queue directly; these resources are managed by the lower layers [6].

4.1 CAN_Frame_t

`CAN_Frame_t` represents a Classical CAN frame at the application level. The fields used in this lab are listed below.

Field	Description
<code>id</code>	CAN identifier of the message.
<code>format</code>	Identifier format: <code>CAN_STANDARD</code> or <code>CAN_EXTENDED</code> .
<code>length</code>	Number of payload bytes, from <code>0</code> to <code>8</code> .
<code>data[]</code>	Byte array containing the payload, up to <code>CAN_MAX_DATA_LENGTH</code> .

Table 18.4.1. Fields of `CAN_Frame_t`

This lab uses `id = 0x100`, `format = CAN_STANDARD`, and `length = 1`, with the transmitted value stored in `data[0]`. The identifier `0x100` is selected for this example; it is not a universally assigned vehicle-speed message ID.

4.2 CAN_begin()

CAN_begin

Initializes CAN at the specified bit rate; the initial operating mode is Normal.

Syntax

```
CAN_begin(bitRate);
```

Parameters

Parameter	Type / accepted value	Description
<code>bitRate</code>	CAN bitrate	Transmission bit rate in bits per second; this lab uses <code>500000</code> bit/s.

Return value

`true` if initialization succeeds; `false` if initialization fails or the bit rate is invalid.

The value `500000UL` corresponds to `500 kbit/s` and is used in NXP's CAN 2.0 example [5].

4.3 CAN_setMode()

CAN_setMode

Selects the CAN controller's operating mode after initialization.

Syntax

```
CAN_setMode(mode);
```

Parameters

Parameter	Type / accepted value	Description
mode	CAN operating mode	This lab uses <code>CAN_MODE_LOOPBACK</code> to receive transmitted frames internally.

Return value

`true` if the mode changes successfully; `false` if the mode is invalid or the controller is not ready.

Both `CAN_begin()` and `CAN_setMode()` must succeed before the program attempts to transmit data [6].

4.4 CAN_send()

CAN_send

Transmits a CAN data frame using blocking operation, waiting for completion or a driver timeout.

Syntax

```
CAN_send(&txFrame);
```

Parameters

Parameter	Type / accepted value	Description
frame	CAN Frame	Address of the <code>CAN_Frame_t</code> containing the frame to transmit.

Return value

`true` if transmission completes successfully; `false` if transmission fails or the frame is invalid.

A valid frame must use an identifier within the range permitted by its `format`, and its `length` must not exceed 8 bytes [6].

4.5 CAN_available() and CAN_read()

CAN_available

Checks whether at least one received frame is present in the CAN receive queue.

Syntax

```
CAN_available();
```

Return value

`true` if a frame is available; `false` if no data is available or CAN is not initialized.

CAN_read

Removes and reads the oldest frame from the receive queue without blocking.

Syntax

```
CAN_read(&rxFrame);
```

Parameters

Parameter	Type / accepted value	Description
<code>frame</code>	CAN Frame	Address of the <code>CAN_Frame_t</code> variable that receives the frame.

Return value

`true` if a frame is read successfully; `false` if the queue is empty or the argument is invalid.

EduFramework stores received data in a queue using interrupt-driven reception. The application checks `CAN_available()` and then uses `CAN_read()` to retrieve and process frames in the main loop [6].

5 Lab Exercise

5.1 Requirements

Develop a CAN Loopback program on one S32K144 board. Initialize CAN at `500 kbit/s` and switch to `CAN_MODE_LOOPBACK`. Every second, the program transmits a Standard CAN frame with ID `0x100`, DLC `1`, and an incrementing data byte.

After transmission, the program reads the returned frame and displays `TX`, `RX`, and `PASS` when the CAN ID, format, DLC, and payload byte match. The Serial Monitor operates at `9600 baud`.

5.2 Program

Use the following program in `src/main.c`, as verified on the MaaZEDU Development Board:

```
#include "Arduino.h"
#include "can.h"
int main(void)
{
    CAN_Frame_t txFrame = {0};
    CAN_Frame_t rxFrame = {0};
    uint8_t value = 0U;
    setup();
    Serial1_begin(9600U);
    if ((false == CAN_begin(500000UL)) ||
        (false == CAN_setMode(CAN_MODE_LOOPBACK)))
    {
        Serial1_println("CAN initialization failed.");
        while (1) {}
    }
    Serial1_println("=== CAN Loopback Demo ===");
    txFrame.id = 0x100UL;
    txFrame.format = CAN_STANDARD;
    txFrame.length = 1U;
    while (1)
    {
        txFrame.data[0] = value;
        if (true == CAN_send(&txFrame))
        {
            Serial1_print("TX: ");
            Serial1_printInt(value);
            Serial1_println("");
            if (true == CAN_available())
            {
                if (true == CAN_read(&rxFrame))
                {
                    Serial1_print("RX: ");
                    Serial1_printInt(rxFrame.data[0]);
                    Serial1_println("");
                    if ((txFrame.id == rxFrame.id) && (txFrame.format == rxFrame.format) &&
                        (txFrame.length == rxFrame.length) &&
                        (txFrame.data[0] == rxFrame.data[0]))
                    {
                        Serial1_println("Result: PASS");
                    }
                    else
                    {
                        Serial1_println("Result: FAIL");
                    }
                }
            }
        }
        else
        {
            Serial1_println("CAN transmission failed.");
        }
        Serial1_println("-----");
        value++;
        delay(1000U);
    }
}
```

Listing 18.5.1. CAN transmission and reception in Internal Loopback mode

`setup()` initializes the EduFramework environment, and `Serial1_begin(9600U)` prepares the Serial Monitor. After `CAN_begin(500000UL)` completes, the program calls `CAN_setMode(CAN_MODE_LOOPBACK)` to receive transmitted frames internally.

`txFrame` is configured with Standard CAN ID `0x100` and a one-byte payload. Within the loop, `value` is assigned to `txFrame.data[0]` and transmitted using `CAN_send()`. If transmission succeeds and a frame is available in the queue, `CAN_read()` stores it in `rxFrame`. The program compares `id`, `format`, `length`, and `data[0]` to determine the `PASS` or `FAIL` result.

The variable `value` ranges from `0` to `255` and increments on each loop iteration. After `255`, the value wraps to `0` because it is an unsigned 8-bit integer. `delay(1000U)` introduces a one-second pause between consecutive tests.

5.3 Verification

After setting the jumper to external-power mode and connecting the `12V` adapter, upload the program to the MaaZEDU Development Board. Open the Serial Monitor at `9600` baud and observe the TX/RX values in successive cycles.

The following output was verified on the hardware:

```
=== CAN Loopback Demo ===
TX: 0
RX: 0
Result: PASS
-----
TX: 1
RX: 1
Result: PASS
-----
TX: 2
RX: 2
Result: PASS
-----
```

Continue monitoring subsequent cycles to verify that the data value increases consistently and the compared fields remain equal. In this program, if no frame is available at the exact time `CAN_available()` is called, the corresponding cycle will not display `RX` or `PASS`. Verification therefore relies on frames that have actually been read and compared, rather than on successful transmission alone.

EXPECTED RESULT

FlexCAN0 initializes successfully and enters Internal Loopback mode. For each received frame, CAN ID `0x100`, Standard format, DLC `1`, and the RX payload byte match the TX values. The Serial Monitor displays `Result: PASS` over consecutive cycles without an external CAN bus connection.

6 Extension

The main exercise uses blocking transmission and polls the receive queue for new data. EduFramework also provides APIs and operating modes for more advanced CAN applications [6].

API / Mode	Function
<code>CAN_sendNonBlocking()</code>	Starts a CAN transmission without waiting for the operation to finish.
<code>CAN_isTxBusy()</code>	Checks whether a non-blocking transmission is currently active.
<code>CAN_isTxComplete()</code>	Checks the successful-completion flag for the most recently started transmission.
<code>CAN_onReceive()</code>	Registers a callback invoked in interrupt context after a frame enters the receive queue.
<code>CAN_MODE_NORMAL</code>	Enables normal CAN communication over the physical bus.
<code>CAN_MODE_LISTEN_ONLY</code>	Monitors CAN traffic without actively transmitting frames.
<code>CAN_EXTENDED</code>	Selects the extended 29-bit CAN identifier format.
<code>CAN_end()</code>	Stops CAN operation and resets the CAN API layer state.

Table 18.6.1. Advanced CAN features available in EduFramework

6.1 Extension Exercise

Modify the transmitted frame to carry two data bytes instead of one. After each transmission, verify both received payload bytes before displaying `PASS`. Keep Internal Loopback mode enabled; no additional hardware is needed.

Non-blocking transmission, receive callbacks, and Normal mode will be used in Lab 19 when two ECUs exchange data over a physical CAN bus.

7 References

- [1] J. A. Cook, J. S. Freudenberger, *Controller Area Network (CAN)*, University of Michigan, EECS 461 Lecture Notes, 2008.
- [2] Steve Corrigan, Texas Instruments, *Introduction to the Controller Area Network (CAN)*, SLOA101B, Rev. B, 2016.
- [3] CAN in Automation (CiA), *CAN CC (Classical CAN)*, CiA CAN Knowledge.
- [4] Kvaser, *The CAN Bus Protocol Tutorial*, Kvaser Technical Tutorial.
- [5] NXP Semiconductors, *S32K1xx Series Cookbook*, AN5413, Rev. 5, 2020.
- [6] EduFramework, *Arduino-style CAN API (can.h, can.c)*, EduFramework Source Code.
- [7] FPT Software, *MaaZ Edu Development Board User Manual*, Rev. 1.0, 2025.